

CSA1305 - Theory of Computation

An introduction to formal models of computation and the theory behind them.

Guided by:

Dr. Balamanigandan R.

Ebin Antony P

Reg no: 192373011

Deterministic Finite Automata — The Basics

A Deterministic Finite Automaton (DFA) is a simple, rigorous model of computation that processes input symbols one at a time using a finite set of states. It's defined precisely by a 5-tuple:

1

Q — States

Finite set of distinct states the machine can be in.

2

Σ — Alphabet

Finite set of input symbols (the language's building blocks).

3

δ — Transition function

Deterministic mapping $\delta: Q \times \Sigma \rightarrow Q$ governing state changes.

4

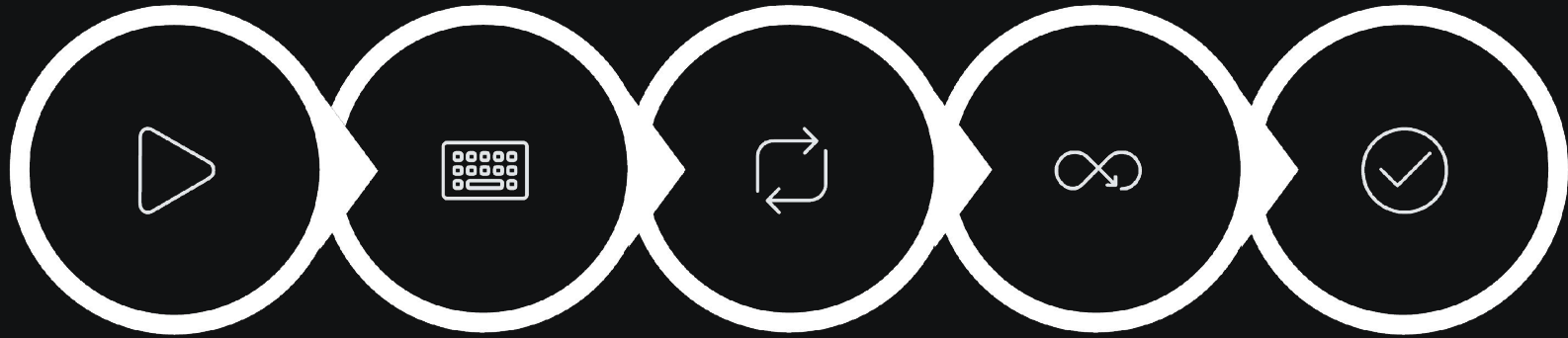
q_0 — Start state

The single state where computation begins.

5

F — Accept states

Subset of Q ; if final state $\in F$ after reading input, the string is accepted.



Start

Read symbol

Transition via δ

Repeat until end

Check final state

Execution steps at a glance:

1. Start at q_0 .
2. Read next input symbol.
3. Use δ to move to exactly one next state (determinism).
4. Repeat until input exhausted.
5. If current state $\in F$, accept; otherwise reject.



Determinism means no branching: for each (state, symbol) pair there is exactly one next state.

Regular Languages, Strengths & Limitations



Recognizes Regular Languages

Every regular language has an equivalent DFA; DFAs provide an exact characterization.



Fast, predictable

Linear-time processing with constant memory (the state register).



Useful for pattern matching

Lexers, simple protocol validators, and hardware controllers commonly use DFAs.

Limitations

- Cannot count arbitrarily (e.g., language $\{ a^n b^n \mid n \geq 0 \}$ is not regular).
- No unbounded memory or nested matching capability — stacks or auxiliary memory required.
- Some languages require nondeterminism or richer models (PDA, Turing machines).

Understanding DFAs lays the groundwork for Pushdown Automata and Turing Machines — each model expands computational power by adding structured memory or unbounded resources.

- ❑ Key takeaway: DFAs = precise, efficient recognizers for regular patterns, but limited when problems require counting or nested dependencies.